

7.8.2 Splines

Để fit regression splines, ta dùng các transform từ package ISLP. Các spline evaluation functions thực sự nằm trong package `scipy.interpolate`; package ISLP chỉ wrap chúng thành các transform tương tự `Poly()` và `PCA()`.

Ở Section 7.4, ta đã thấy regression splines có thể được fit bằng cách construct một matrix basis functions phù hợp. Function `BSpline()` tạo toàn bộ matrix basis functions cho spline với set of knots được chỉ định. Mặc định các B-splines được tạo là cubic; để đổi degree, dùng argument `degree`.

BSpline()

In [16]:

```
bs_ = BSpline(internal_knots=[25,40,60], intercept=True).fit(age)
bs_age = bs_.transform(age)
bs_age.shape
```

Out[16]:

```
(3000, 7)
```

Kết quả là một matrix bảy columns, đúng như kỳ vọng với cubic-spline basis có 3 interior knots. Ta có thể tạo cùng matrix này bằng object `bs()`, thuận tiện cho việc đưa basis vào model-matrix builder (tương tự quan hệ giữa `poly()` và workhorse `Poly()`) đã mô tả trong Section 7.8.1.

Bây giờ ta fit cubic spline model cho data `Wage`.

In [17]:

```
bs_age = MS([bs('age', internal_knots=[25,40,60])])
Xbs = bs_age.fit_transform(Wage)
M = sm.OLS(y, Xbs).fit()
summarize(M)
```

Out[17]:

	coef	std err	...
intercept	60.494	9.460	...
bs(age, internal_knots=[25, 40, 60])[0]	3.980	12.538	...
bs(age, internal_knots=[25, 40, 60])[1]	44.631	9.626	...
bs(age, internal_knots=[25, 40, 60])[2]	62.839	10.755	...
bs(age, internal_knots=[25, 40, 60])[3]	55.991	10.706	...
bs(age, internal_knots=[25, 40, 60])[4]	50.688	14.402	...
bs(age, internal_knots=[25, 40, 60])[5]	16.606	19.126	...

Column names hơi cồng kềnh, khiến printed summary ở trên phải truncate. Ta có thể đặt tên ngay khi construct bằng argument `name` như sau.

In [18]:

```
bs_age = MS([bs('age',
                internal_knots=[25,40,60],
                name='bs(age)')])
Xbs = bs_age.fit_transform(Wage)
M = sm.OLS(y, Xbs).fit()
summarize(M)
```

Out[18]:

	coef	std err	t	P> t
intercept	60.494	9.460	6.394	0.000
bs(age, knots)[0]	3.981	12.538	0.317	0.751
bs(age, knots)[1]	44.631	9.626	4.636	0.000
bs(age, knots)[2]	62.839	10.755	5.843	0.000
bs(age, knots)[3]	55.991	10.706	5.230	0.000
bs(age, knots)[4]	50.688	14.402	3.520	0.000
bs(age, knots)[5]	16.606	19.126	0.868	0.385

Chú ý có 6 spline coefficients chứ không phải 7. Nguyên nhân là mặc định

`bs()` giả định `intercept=False`, vì model thường đã có overall intercept. Do đó nó generate spline basis với các knots đã cho rồi discard một basis function để account cho intercept.

Ta cũng có thể dùng option `df` (degrees of freedom) để chỉ định complexity của spline. Ở trên, 3 knots tạo spline basis có 6 columns hay 6 degrees of freedom. Nếu chỉ định `df=6` thay vì actual knots, `bs()` sẽ tạo spline với 3 knots đặt tại uniform quantiles của training data. Có thể xem các knots được chọn trực tiếp bằng `BSpline()`:

```
In [19]: BSpline(df=6).fit(age).internal_knots_
```

```
Out[19]: array([33.75, 42.0, 51.0])
```

Khi yêu cầu sáu degrees of freedom, transform chọn knots tại ages 33.75, 42.0 và 51.0, tương ứng các percentiles 25th, 50th và 75th của `age`.

Với B-splines, ta không bắt buộc phải dùng cubic polynomials (tức `degree=3`). Chẳng hạn, `degree=0` tạo piecewise constant functions như example dùng `pd.qcut()` ở trên.

```
In [20]: bs_age0 = MS([bs('age',
                        df=3,
                        degree=0)]).fit(Wage)
Xbs0 = bs_age0.transform(Wage)
summarize(sm.OLS(y, Xbs0).fit())
```

```
Out[20]:
```

	coef	std err	t	P> t
intercept	94.158	1.478	63.687	0.0
bs(age, df=3, degree=0)[0]	22.349	2.152	10.388	0.0
bs(age, df=3, degree=0)[1]	24.808	2.044	12.137	0.0
bs(age, df=3, degree=0)[2]	22.781	2.087	10.917	0.0

Fit này nên được compare với cell [15], nơi `qcut()` tạo bốn bins bằng cách cắt tại 25%, 50% và 75% quantiles của `age`. Vì ở đây ta chỉ định `df=3` cho degree-zero splines, cũng sẽ có knots tại đúng ba quantiles đó. Dù coefficients trông khác nhau, đây chỉ là hệ quả của coding khác nhau. Chẳng hạn first coefficient giống hệt ở cả hai case và chính là mean response trong first bin. Với second coefficient, $94.158 + 22.349 = 116.507 \approx 116.611$, trong đó giá trị sau là mean ở second bin trong cell [15]. Ở đây intercept được code bằng một column of ones, nên second, third và fourth coefficients là increments cho các bins tương ứng. Tổng không hoàn toàn giống nhau vì `qcut()` dùng `≤`, còn `bs()` dùng `<` khi quyết định bin membership.

Để fit natural spline, ta dùng transform `NaturalSpline()` với helper tương ứng `ns()`. Ở đây ta fit natural spline với năm degrees of freedom (không tính intercept) và plot kết quả.

NaturalSpline()

```
In [21]: ns_age = MS([ns('age', df=5)]).fit(Wage)
M_ns = sm.OLS(y, ns_age.transform(Wage)).fit()
summarize(M_ns)
```

```
Out[21]:
```

	coef	std err	t	P> t
intercept	60.475	4.708	12.844	0.000
ns(age, df=5)[0]	61.527	4.709	13.065	0.000
ns(age, df=5)[1]	55.691	5.717	9.741	0.000
ns(age, df=5)[2]	46.818	4.948	9.463	0.000

```
ns(age, df=5)[3]    83.204    11.918    6.982    0.000
ns(age, df=5)[4]     6.877     9.484     0.725    0.468
```

Bây giờ plot natural spline bằng plotting function đã viết trước đó.

In [22]:

```
plot_wage_fit(age_df,
               ns_age,
               'Natural spline, df=5');
```

7.8.3 Smoothing Splines and GAMs

Smoothing spline là một special case của GAM với squared-error loss và một feature. Để fit GAMs trong Python, ta dùng package `pygam`, có thể install bằng `pip install pygam`. Estimator `LinearGAM()` dùng squared-error loss. GAM được specified bằng cách associate mỗi column của model matrix với một smoothing operation cụ thể: `s` cho smoothing spline, `l` cho linear, và `f` cho factor hay categorical variables. Argument 0 truyền vào `s` bên dưới cho biết smoother sẽ apply vào first column của feature matrix. Ở đây ta truyền matrix chỉ có một column là `X_age`. Argument `lam` là penalty parameter λ như Section 7.5.2.

`pygam`
`LinearGAM()`
In [23]:

```
X_age = np.asarray(age).reshape((-1,1))
gam = LinearGAM(s_gam(0, lam=0.6))
gam.fit(X_age, y)
```

Out[23]:

```
LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
           max_iter=100, scale=None, terms=s(0) + intercept, tol=0.0001,
           verbose=False)
```

Library `pygam` thường kỳ vọng một matrix of features, nên ta reshape `age` thành matrix (two-dimensional array) thay vì vector (one-dimensional array). Giá trị `-1` trong `reshape()` bảo numpy suy ra size của dimension đó từ các entry còn lại của shape tuple.

Hãy xem fit thay đổi thế nào theo smoothing parameter `lam`. Function `np.logspace()` giống `np.linspace()` nhưng spacing đều trên log-scale. Dưới đây ta thay đổi `lam` từ 10^{-2} đến 10^6 .

`np.logspace()`

In [24]:

```
fig, ax = subplots(figsize=(8,8))
ax.scatter(age, y, facecolor='gray', alpha=0.5)
for lam in np.logspace(-2, 6, 5):
    gam = LinearGAM(s_gam(0, lam=lam)).fit(X_age, y)
    ax.plot(age_grid,
            gam.predict(age_grid),
            label='{:.1e}'.format(lam),
            linewidth=3)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='$\\lambda$');
```

Package `pygam` có thể search optimal smoothing parameter.

In [25]:

```
gam_opt = gam.gridsearch(X_age, y)
ax.plot(age_grid,
        gam_opt.predict(age_grid),
        label='Grid search',
        linewidth=4)
ax.legend()
```

fig

Một alternative là fix degrees of freedom của smoothing spline bằng function trong package ISLP `pygam`. Bên dưới ta tìm một λ cho roughly four degrees of freedom. Lưu ý các degrees of freedom này bao gồm unpenalized intercept và linear term của smoothing spline, vì thế luôn có ít nhất hai degrees of freedom.

```
In [26]: age_term = gam.terms[0]
lam_4 = approx_lam(X_age, age_term, 4)
age_term.lam = lam_4
degrees_of_freedom(X_age, age_term)
```

```
Out[26]: 4.000000100004728
```

Ta thay đổi degrees of freedom trên plot tương tự phía trên. Ta chọn degrees of freedom bằng desired degrees of freedom cộng một để account cho fact rằng các smoothing splines này luôn có intercept term. Vì vậy giá trị $df=1$ tương ứng linear fit.

```
In [27]: fig, ax = subplots(figsize=(8,8))
ax.scatter(X_age,
           y,
           facecolor='gray',
           alpha=0.3)
for df in [1,3,4,8,15]:
    lam = approx_lam(X_age, age_term, df+1)
    age_term.lam = lam
    gam.fit(X_age, y)
    ax.plot(age_grid,
            gam.predict(age_grid),
            label='{ :d}'.format(df),
            linewidth=4)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='Degrees of freedom');
```

Additive Models with Several Terms

Điểm mạnh của generalized additive models là khả năng fit multivariate regression models flexible hơn linear models. Ta sẽ minh họa hai approach: một approach manual hơn dùng natural splines và piecewise constant functions, và approach thứ hai dùng package `pygam` cùng smoothing splines.

Trước hết ta fit GAM bằng tay để predict `wage` từ natural spline functions của `year` và `age`, đồng thời treat `education` như qualitative predictor, như trong (7.16). Vì đây đơn giản là một linear regression model lớn dùng basis functions thích hợp, ta có thể fit trực tiếp bằng `sm.OLS()`.

Ta build model matrix thủ công hơn ở đây, vì muốn truy cập riêng từng piece khi construct partial dependence plots.

```
In [28]: ns_age = NaturalSpline(df=4).fit(age)
ns_year = NaturalSpline(df=5).fit(Wage['year'])
Xs = [ns_age.transform(age),
      ns_year.transform(Wage['year']),
      pd.get_dummies(Wage['education']).values]
X_bh = np.hstack(Xs)
gam_bh = sm.OLS(y, X_bh).fit()
```

Ở đây `NaturalSpline()` là workhorse đứng sau helper `ns()`. Ta dùng toàn bộ columns của indicator matrix cho categorical variable `education`, khiến intercept trở nên redundant. Cuối cùng, ta stack ba component matrices theo chiều ngang để tạo model matrix `X_bh`.

Bây giờ ta construct partial dependence plots cho từng term trong rudimentary GAM này. Có thể làm thủ công nếu có grids cho `age` và `year`: predict bằng các new `X` matrices trong đó chỉ thay đổi một feature tại một thời điểm, còn lại fix.

In [29]:

```
age_grid = np.linspace(age.min(),
                        age.max(),
                        100)
X_age_bh = X_bh.copy()[1:100]
X_age_bh[:, :] = X_bh[:, :].mean(0)[None, :]
X_age_bh[:, :4] = ns_age.transform(age_grid)
preds = gam_bh.get_prediction(X_age_bh)
bounds_age = preds.conf_int(alpha=0.05)
partial_age = preds.predicted_mean
center = partial_age.mean()
partial_age -= center
bounds_age -= center
fig, ax = subplots(figsize=(8,8))
ax.plot(age_grid, partial_age, 'b', linewidth=3)
ax.plot(age_grid, bounds_age[:,0], 'r--', linewidth=3)
ax.plot(age_grid, bounds_age[:,1], 'r--', linewidth=3)
ax.set_xlabel('Age')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of age on wage', fontsize=20);
```

Chi tiết các bước vừa làm như sau. Ý tưởng là tạo một new prediction matrix, trong đó mọi column không thuộc `age` đều constant và set bằng training-data means, còn bốn columns của `age` được fill bằng natural spline basis evaluated tại 100 values trong `age_grid`.

1. Tạo grid length 100 theo `age`, và matrix `X_age_bh` có 100 rows cùng số columns như `X_bh`.
2. Replace mọi row của matrix bằng column means của original matrix.
3. Replace riêng bốn columns đầu, đại diện cho `age`, bằng natural spline basis tính tại các values trong `age_grid`.

Các bước còn lại đã quen thuộc.

Ta cũng xem effect của `year` lên `wage`; process giống hệt.

In [30]:

```
year_grid = np.linspace(2003, 2009, 100)
year_grid = np.linspace(Wage['year'].min(),
                        Wage['year'].max(),
                        100)
X_year_bh = X_bh.copy()[1:100]
X_year_bh[:, :] = X_bh[:, :].mean(0)[None, :]
X_year_bh[:, :4:9] = ns_year.transform(year_grid)
preds = gam_bh.get_prediction(X_year_bh)
bounds_year = preds.conf_int(alpha=0.05)
partial_year = preds.predicted_mean
center = partial_year.mean()
partial_year -= center
bounds_year -= center
fig, ax = subplots(figsize=(8,8))
ax.plot(year_grid, partial_year, 'b', linewidth=3)
ax.plot(year_grid, bounds_year[:,0], 'r--', linewidth=3)
ax.plot(year_grid, bounds_year[:,1], 'r--', linewidth=3)
```

```
ax.set_xlabel('Year')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of year on wage', fontsize=20);
```

Tiếp theo ta fit model (7.16) bằng smoothing splines thay vì natural splines. Mọi term trong (7.16) được fit simultaneously, nên mỗi term tính đến các term khác khi explain response. Package `pygam` chỉ làm việc với matrices, vì vậy categorical series `education` phải được convert sang array representation qua attribute `cat.codes`. Vì `year` chỉ có 7 unique values, ta chỉ dùng bảy basis functions cho term này.

In [31]:

```
gam_full = LinearGAM(s_gam(0) +
                    s_gam(1, n_splines=7) +
                    f_gam(2, lam=0))
Xgam = np.column_stack([age,
                        Wage['year'],
                        Wage['education'].cat.codes])
gam_full = gam_full.fit(Xgam, y)
```

Hai terms `s_gam()` tạo smoothing spline fits và dùng default λ (`lam=0.6`), một lựa chọn khá arbitrary. Với categorical term `education`, specified bằng `f_gam()`, ta đặt `lam=0` để tránh shrinkage. Ta tạo partial dependence plot theo `age` để xem effect của các lựa chọn này.

Các values cho plot được generate bởi package `pygam`. Package `ISLP.pygam` cung cấp function `plot_gam()` cho partial-dependence plots, giúp việc này dễ hơn example natural spline vừa làm.

plot_gam()

In [32]:

```
fig, ax = subplots(figsize=(8,8))
plot_gam(gam_full, 0, ax=ax)
ax.set_xlabel('Age')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of age on wage - default lam=0.6',
            fontsize=20);
```

Function khá wiggly. Tự nhiên hơn là specify `df` thay vì `lam`. Ta refit GAM dùng four degrees of freedom cho cả `age` và `year`. Nhớ rằng cộng một bên dưới là để account cho intercept của smoothing spline.

In [33]:

```
age_term = gam_full.terms[0]
age_term.lam = approx_lam(Xgam, age_term, df=4+1)
year_term = gam_full.terms[1]
year_term.lam = approx_lam(Xgam, year_term, df=4+1)
gam_full = gam_full.fit(Xgam, y)
```

Chú ý: update `age_term.lam` ở trên cũng update term tương ứng trong `gam_full`; cụ thể là `gam_full.terms[0]`. Tương tự cho `year_term.lam`.

Lặp lại plot cho `age` cho thấy curve smooth hơn nhiều. Ta cũng plot `year`.

In [34]:

```
fig, ax = subplots(figsize=(8,8))
plot_gam(gam_full,
        1,
        ax=ax)
ax.set_xlabel('Year')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of year on wage', fontsize=20)
```

Cuối cùng plot `education`, một categorical variable. Partial dependence plot có dạng khác và phù hợp hơn với set of fitted constants cho từng level.

```
In [35]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_full, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of wage on education',
             fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```

ANOVA Tests for Additive Models

Trong mọi model, function của `year` trông gần như linear. Ta có thể thực hiện series ANOVA tests để xác định model nào tốt nhất trong ba model: GAM bỏ `year` (M_1), GAM dùng linear function của `year` (M_2), hoặc GAM dùng spline function của `year` (M_3).

```
In [36]: gam_0 = LinearGAM(age_term + f_gam(2, lam=0))
gam_0.fit(Xgam, y)
gam_linear = LinearGAM(age_term +
                       l_gam(1, lam=0) +
                       f_gam(2, lam=0))
gam_linear.fit(Xgam, y)
```

```
Out[36]: LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
                 max_iter=100, scale=None, terms=s(0) + l(1) + f(2) + intercept,
                 tol=0.0001, verbose=False)
```

Chú ý việc dùng `age_term` trong expressions trên. Ta làm vậy vì trước đó đã set `lam` trong term này để đạt four degrees of freedom.

Để trực tiếp assess effect của `year`, ta chạy ANOVA trên ba fitted models.

```
In [37]: anova_gam(gam_0, gam_linear, gam_full)
```

```
Out[37]:
```

	deviance	df	deviance_diff	df_diff	F	pvalue
0	3714362.366	2991.004	NaN	NaN	NaN	NaN
1	3696745.823	2990.005	17616.543	0.999	14.265	0.002
2	3693142.930	2987.007	3602.894	2.998	0.972	0.436

Có compelling evidence rằng GAM với linear function theo `year` tốt hơn GAM bỏ `year` hoàn toàn ($p\text{-value} = 0.002$). Tuy nhiên, không có evidence rằng cần non-linear function của `year` ($p\text{-value} = 0.435$). Nói cách khác, từ ANOVA này M_2 được preferred.

Ta lặp process tương tự cho `age`. Kết quả cho evidence rất rõ rằng cần một non-linear term cho `age`.

```
In [38]: gam_0 = LinearGAM(year_term +
                           f_gam(2, lam=0))
gam_linear = LinearGAM(l_gam(0, lam=0) +
                       year_term +
                       f_gam(2, lam=0))
gam_0.fit(Xgam, y)
gam_linear.fit(Xgam, y)
anova_gam(gam_0, gam_linear, gam_full)
```

```
Out[38]:
```

	deviance	df	deviance_diff	df_diff	F	pvalue
0	3975443.045	2991.001	NaN	NaN	NaN	NaN
1	3850246.908	2990.001	125196.137	1.000	101.270	0.000
2	3693142.930	2987.007	157103.978	2.993	42.448	0.000

GAM fit có method `summary()` khá verbose; sách không reproduce output ở đây.

```
In [39]: gam_full.summary()
```

Ta có thể predict từ `gam` objects giống `lm` objects, dùng method `predict()` của class `gam`. Ở đây ta predict trên training set.

```
In [40]: Yhat = gam_full.predict(Xgam)
```

Để fit logistic regression GAM, ta dùng `LogisticGAM()` từ `pygam`.

LogisticGAM()

```
In [41]: gam_logit = LogisticGAM(age_term +
                                l_gam(1, lam=0) +
                                f_gam(2, lam=0))
gam_logit.fit(Xgam, high_earn)
```

```
Out[41]: LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
                        fit_intercept=True, max_iter=100,
                        terms=s(0) + l(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

```
In [42]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of wage on education',
             fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```

Model trông khá flat, với error bars đặc biệt lớn ở first category. Hãy xem data kỹ hơn.

```
In [43]: pd.crosstab(Wage['high_earn'], Wage['education'])
```

Không có high earners ở first category của `education`, nên model sẽ rất khó fit. Ta sẽ fit logistic regression GAM sau khi exclude tất cả observations thuộc category này; kết quả hợp lý hơn.

Ta có thể subset model matrix, nhưng như vậy column tương ứng vẫn còn trong `Xgam`. Dù có thể suy ra column nào ứng với feature này, để reproducible ta reform model matrix trên subset nhỏ hơn.

```
In [44]: only_hs = Wage['education'] == '1. < HS Grad'
Wage_ = Wage.loc[~only_hs]
Xgam_ = np.column_stack([Wage_['age'],
                          Wage_['year'],
                          Wage_['education'].cat.codes-1])
high_earn_ = Wage_['high_earn']
```

Ở second-to-last line bên trên, ta subtract one khỏi category codes vì một bug trong `pygam`. Việc này chỉ relabel education values nên không ảnh

hưởng fit.

Bây giờ fit model.

```
In [45]: gam_logit_ = LogisticGAM(age_term +
                                year_term +
                                f_gam(2, lam=0))
gam_logit_.fit(Xgam_, high_earn_)
```

```
Out[45]: LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
                      fit_intercept=True, max_iter=100,
                      terms=s(0) + s(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

Hãy xem effect của education, year và age lên high-earner status sau khi đã remove các observations nói trên.

```
In [46]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of high earner status on education',
             fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories[1:],
                  fontsize=8);
```

```
In [47]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 1)
ax.set_xlabel('Year')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of high earner status on year',
             fontsize=20);
```

```
In [48]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 0)
ax.set_xlabel('Age')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of high earner status on age',
             fontsize=20);
```

7.8.4 Local Regression

Ta minh họa local regression bằng function `lowess()`. Function này nằm trong module `sm.nonparametric`. Một số implementation của GAM cho phép terms là local regression operators; `pygam` thì không.

Ở đây ta fit local linear regression models với spans 0.2 và 0.5; tức mỗi neighborhood gồm 20% hoặc 50% observations. Như kỳ vọng, span 0.5 cho curve smooth hơn 0.2.

```
In [49]: lowess = sm.nonparametric.lowess
fig, ax = subplots(figsize=(8,8))
ax.scatter(age, y, facecolor='gray', alpha=0.5)
for span in [0.2, 0.5]:
    fitted = lowess(y,
                    age,
                    frac=span,
                    xvals=age_grid)
    ax.plot(age_grid,
            fitted,
```

lowess()

```
label='{:.1f}'.format(span),
linewidth=4)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='span', fontsize=15);
```

7.9 Exercises

Conceptual



- Trong chapter này đã nói rằng cubic regression spline với một knot tại ξ có thể được tạo bằng basis $x, x^2, x^3, (x-\xi)_+^3$, trong đó $(x-\xi)_+^3 = (x-\xi)^3$ nếu $x > \xi$ và bằng 0 nếu không. Ta sẽ chứng minh một function dạng

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)_+^3$$

thực sự là cubic regression spline, bất kể values của $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$.

- Tìm cubic polynomial

$$f_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3$$

sao cho $f(x) = f_1(x)$ với mọi $x \leq \xi$. Biểu diễn a_1, b_1, c_1, d_1 theo $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$.

- Tìm cubic polynomial

$$f_2(x) = a_2 + b_2 x + c_2 x^2 + d_2 x^3$$

sao cho $f(x) = f_2(x)$ với mọi $x > \xi$. Biểu diễn a_2, b_2, c_2, d_2 theo $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$. Như vậy ta đã establish rằng $f(x)$ là piecewise polynomial.

- Chứng minh $f_1(\xi) = f_2(\xi)$; tức $f(x)$ continuous tại ξ .
- Chứng minh $f'_1(\xi) = f'_2(\xi)$; tức $f'(x)$ continuous tại ξ .
- Chứng minh $f''_1(\xi) = f''_2(\xi)$; tức $f''(x)$ continuous tại ξ .

Do đó $f(x)$ thực sự là cubic spline.

Hint: Parts (d) và (e) cần kiến thức single-variable calculus. Nhắc lại, với cubic polynomial

$$f_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3,$$

first derivative là

$$f'_1(x) = b_1 + 2c_1 x + 3d_1 x^2,$$

và second derivative là

$$f''_1(x) = 2c_1 + 6d_1 x.$$

- Giả sử một curve $\hat{g}$ được compute để smoothly fit một set gồm n points bằng formula

$$\hat{g} = \arg \min_g \left\{ \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int [g^{(m)}(x)]^2 dx \right\},$$

trong đó $g^{(m)}$ là m th derivative của g (và $g^{(0)} = g$). Hãy sketch example của $\hat{g}$ trong các scenario sau.

- (a) $\lambda = \infty, m = 0$.
 (b) $\lambda = \infty, m = 1$.
 (c) $\lambda = \infty, m = 2$.
 (d) $\lambda = \infty, m = 3$.
 (e) $\lambda = 0, m = 3$.
3. Giả sử ta fit một curve với basis functions $b_1(X) = X$, $b_2(X) = (X - 1)^2 I(X \geq 1)$, trong đó $I(X \geq 1)$ bằng 1 nếu $X \geq 1$ và 0 nếu không. Ta fit linear regression model

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon,$$

và thu được coefficient estimates $\hat{\beta}_0 = 1$, $\hat{\beta}_1 = 1$, $\hat{\beta}_2 = -2$. Hãy sketch estimated curve từ $X = -2$ đến $X = 2$, ghi rõ intercepts, slopes và các thông tin liên quan khác.

4. Giả sử ta fit một curve với basis functions

$$b_1(X) = I(0 \leq X \leq 2) - (X - 1)I(1 \leq X \leq 2),$$

$$b_2(X) = (X - 3)I(3 \leq X \leq 4) + I(4 < X \leq 5).$$

Ta fit linear regression model

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon,$$

và thu được $\hat{\beta}_0 = 1$, $\hat{\beta}_1 = 1$, $\hat{\beta}_2 = 3$. Hãy sketch estimated curve từ $X = -2$ đến $X = 6$, ghi rõ intercepts, slopes và các thông tin liên quan khác.

5. Xét hai curves $\hat{g}_1$ và $\hat{g}_2$ được định nghĩa bởi

$$\hat{g}_1 = \arg \min_g \left\{ \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int [g^{(3)}(x)]^2 dx \right\},$$

$$\hat{g}_2 = \arg \min_g \left\{ \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int [g^{(4)}(x)]^2 dx \right\},$$

trong đó $g^{(m)}$ là m th derivative của g .

- (a) Khi $\lambda \rightarrow \infty$, $\hat{g}_1$ hay $\hat{g}_2$ có training RSS nhỏ hơn?
 (b) Khi $\lambda \rightarrow \infty$, $\hat{g}_1$ hay $\hat{g}_2$ có test RSS nhỏ hơn?
 (c) Với $\lambda = 0$, $\hat{g}_1$ hay $\hat{g}_2$ có training và test RSS nhỏ hơn?

Applied

6. Trong exercise này, hãy phân tích thêm data set **Wage** đã dùng xuyên suốt chapter.
- (a) Thực hiện polynomial regression để predict **wage** bằng **age**. Dùng cross-validation chọn optimal degree d của polynomial. Degree nào được chọn, và kết quả này so với hypothesis testing dùng ANOVA ra sao? Vẽ plot của polynomial fit thu được trên data.

- (b) Fit step function để predict **wage** bằng **age**, và dùng cross-validation chọn optimal number of cuts. Vẽ plot của fit thu được.
7. Data set **Wage** chứa nhiều feature khác chưa được explore trong chapter, như marital status (**maritl**), job class (**jobclass**) và các variable khác. Hãy explore relationship giữa một số predictor này với **wage**, rồi dùng non-linear fitting techniques để fit flexible models. Tạo plots của kết quả và viết summary về findings.
 8. Fit một số non-linear models đã khảo sát trong chapter cho data set **Auto**. Có evidence về non-linear relationships trong data set này hay không? Tạo các informative plots để justify câu trả lời.
 9. Câu hỏi này dùng variables **dis** (weighted mean của distances tới năm Boston employment centers) và **nox** (nitrogen oxides concentration theo parts per 10 million) từ data **Boston**. Ta treat **dis** là predictor và **nox** là response.
 - (a) Dùng function **poly()** từ module **ISLP.models** để fit cubic polynomial regression predict **nox** từ **dis**. Report regression output và plot data cùng polynomial fit.
 - (b) Plot polynomial fits cho một range các polynomial degrees (chẳng hạn 1 đến 10), và report associated residual sum of squares.
 - (c) Dùng cross-validation hoặc approach khác để chọn optimal degree cho polynomial, rồi giải thích kết quả.
 - (d) Dùng function **bs()** từ module **ISLP.models** để fit regression spline predict **nox** từ **dis**. Report output cho fit với four degrees of freedom. Bạn chọn knots thế nào? Plot fit thu được.
 - (e) Tiếp theo fit regression spline cho một range degrees of freedom, plot các fits và report RSS tương ứng. Mô tả results.
 - (f) Dùng cross-validation hoặc approach khác để chọn best degrees of freedom cho regression spline trên data này. Mô tả results.
 10. Câu hỏi này liên quan đến data set **College**.
 - (a) Chia data thành training set và test set. Dùng out-of-state tuition làm response và các variables còn lại làm predictors, thực hiện forward stepwise selection trên training set để identify một satisfactory model chỉ dùng subset predictors.
 - (b) Fit GAM trên training data, với out-of-state tuition là response và các features chọn ở step trước là predictors. Plot results và giải thích findings.
 - (c) Evaluate model thu được trên test set và giải thích results.
 - (d) Với variable nào, nếu có, có evidence về non-linear relationship với response?
 11. Trong Section 7.7 có nói GAMs thường được fit bằng backfitting. Ý tưởng của backfitting thực ra khá đơn giản. Ta sẽ explore backfitting trong context multiple linear regression.
Giả sử ta muốn thực hiện multiple linear regression nhưng không có

software cho việc đó; chỉ có software cho simple linear regression. Ta dùng iterative approach sau: repeatedly giữ fixed mọi coefficient estimate trừ một estimate tại current value, và chỉ update coefficient đó bằng simple linear regression. Process tiếp tục đến convergence, tức đến khi coefficient estimates ngừng thay đổi. Hãy thử trên toy example.

- (a) Generate response Y và hai predictors X_1, X_2 với $n = 100$.
- (b) Viết function `simple_reg()` nhận hai arguments `outcome` và `feature`, fit simple linear regression model với `outcome` và `feature` này, rồi return estimated intercept và slope.
- (c) Initialize `beta1` bằng value tùy ý; choice không quan trọng.
- (d) Giữ `beta1` fixed, dùng `simple_reg()` fit model

$$Y - \text{beta1} \cdot X_1 = \beta_0 + \beta_2 X_2 + \epsilon.$$

Lưu values thu được thành `beta0` và `beta2`.

- (e) Giữ `beta2` fixed, fit model

$$Y - \text{beta2} \cdot X_2 = \beta_0 + \beta_1 X_1 + \epsilon.$$

Lưu kết quả thành `beta0` và `beta1`, overwrite previous values.

- (f) Viết for loop lặp (c) và (d) 1,000 lần. Report estimates của `beta0`, `beta1`, `beta2` ở mỗi iteration. Tạo plot hiển thị từng value của ba estimates này.
 - (g) Compare answer trong (e) với results khi đơn giản dùng multiple linear regression để predict Y bằng X_1 và X_2 . Dùng method `axline()` overlay các multiple linear regression coefficient estimates đó lên plot ở (e).
 - (h) Trên data set này, cần bao nhiêu backfitting iterations để có một “good” approximation cho multiple regression coefficient estimates?
12. Problem này tiếp tục exercise trước. Trong toy example với $p = 100$, hãy chứng minh có thể approximate multiple linear regression coefficient estimates bằng cách repeatedly thực hiện simple linear regression trong backfitting procedure. Cần bao nhiêu backfitting iterations để đạt “good” approximation cho multiple regression coefficient estimates? Tạo plot để justify câu trả lời.

8

Tree-Based Methods

Trong chapter này, ta mô tả tree-based methods cho regression và classification. Các method này stratify hoặc segment predictor space thành một số region đơn giản. Để predict cho một observation, ta thường dùng mean hoặc mode response value của training observations nằm trong region chứa observation đó. Vì set of splitting rules dùng để segment predictor space có thể được summarize bằng một tree, các approach kiểu này được gọi là *decision tree methods*.

decision tree

Tree-based methods đơn giản và hữu ích cho interpretation. Tuy nhiên, xét về prediction accuracy, chúng thường không cạnh tranh được với các supervised learning approaches tốt nhất như những approach ở Chapters 6 và 7. Vì vậy chapter này còn giới thiệu bagging, random forests, boosting và Bayesian additive regression trees. Mỗi approach tạo nhiều trees rồi combine chúng thành một consensus prediction duy nhất. Ta sẽ thấy việc combine số lượng lớn trees thường cải thiện prediction accuracy rất mạnh, đổi lại là một phần interpretability.

8.1 The Basics of Decision Trees

Decision trees có thể dùng cho cả regression lẫn classification problems. Trước hết ta xét regression, rồi chuyển sang classification.

8.1.1 Regression Trees

Để motivate regression trees, ta bắt đầu bằng một simple example.

regression tree

Predicting Baseball Players' Salaries Using Regression Trees

Ta dùng data set **Hitters** để predict **Salary** của một baseball player dựa trên **Years** (số năm chơi ở major leagues) và **Hits** (số hits ở năm trước). Trước hết remove observations thiếu **Salary**, rồi log-transform **Salary** để distribution có dạng bell-shape điển hình hơn. Nhớ rằng **Salary** được đo theo thousands of dollars.

Figure 8.1 cho regression tree fit vào data. Tree gồm series splitting rules bắt đầu từ top. Top split gán observations có **Years**<4.5 vào left branch.¹

¹Cả **Years** và **Hits** đều là integers trong data này; function dùng để fit tree label splits tại



HÌNH 8.1. Với data *Hitters*, regression tree dùng để predict log salary của một baseball player dựa trên số năm anh ta chơi ở major leagues và số hits trong năm trước. Tại một internal node, label dạng $X_j < t_k$ chỉ left-hand branch của split; right-hand branch tương ứng $X_j \geq t_k$. Chẳng hạn top split tạo hai branch lớn: left branch là *Years*<4.5, right branch là *Years*>=4.5. Tree có hai internal nodes và ba terminal nodes, hay leaves. Số trong mỗi leaf là mean response của observations rơi vào leaf đó.

Predicted salary cho những players này bằng mean response của training observations có *Years*<4.5. Mean log salary là 5.107, nên prediction là $e^{5.107}$ thousand dollars, tức \$165,174. Players có *Years*>=4.5 đi sang right branch, rồi group này tiếp tục được subdivide theo *Hits*. Nhìn chung tree stratify/segment players thành ba regions của predictor space: players chơi bốn năm hoặc ít hơn; players chơi ít nhất năm năm và có dưới 118 hits ở năm trước; và players chơi ít nhất năm năm với ít nhất 118 hits ở năm trước. Ba regions là

$$R_1 = \{X \mid \text{Years} < 4.5\}, \quad R_2 = \{X \mid \text{Years} \geq 4.5, \text{Hits} < 117.5\},$$

$$R_3 = \{X \mid \text{Years} \geq 4.5, \text{Hits} \geq 117.5\}.$$

Figure 8.2 minh họa các regions theo *Years* và *Hits*.

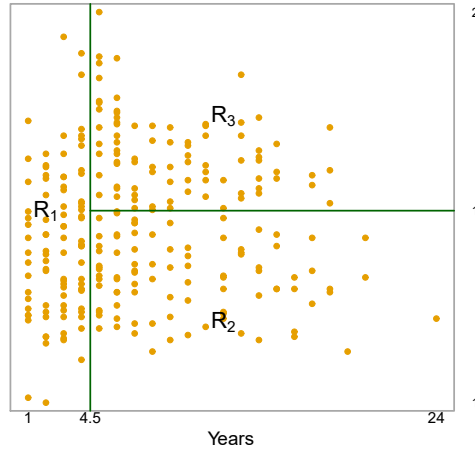
Predicted salaries cho ba group này lần lượt là $\$1,000 \times e^{5.107} = \$165,174$, $\$1,000 \times e^{5.999} = \$402,834$, và $\$1,000 \times e^{6.740} = \$845,346$.

Theo analogy với tree, regions R_1, R_2, R_3 được gọi là *terminal nodes* hay *leaves*. Decision trees thường được vẽ upside down: leaves ở bottom. Các points nơi predictor space bị split gọi là *internal nodes*. Trong Figure 8.1, hai internal nodes được chỉ bằng text *Years*<4.5 và *Hits*<117.5. Các segments nối nodes gọi là *branches*.

Có thể interpret tree ở Figure 8.1 như sau: *Years* là factor quan trọng nhất quyết định *Salary*; players ít experience nhận salary thấp hơn players nhiều experience. Với players ít experience, số hits năm trước dường như đóng vai trò nhỏ. Nhưng với players đã ở major leagues ít nhất năm năm, số hits năm trước ảnh hưởng đến salary, và players có nhiều hits hơn thường nhận salary cao hơn. Regression tree ở Figure 8.1 có lẽ oversimplify true

terminal node
leaf
internal node
branch

midpoint giữa hai adjacent values.



HÌNH 8.2. Three-region partition của data set *Hitters* từ regression tree trong Figure 8.1.

relationship giữa *Hits*, *Years*, và *Salary*, nhưng có advantages so với các regression models khác ở Chapters 3 và 6: dễ interpret và có graphical representation rõ ràng.

Prediction via Stratification of the Feature Space

Bây giờ xét process build regression tree. Roughly speaking có hai steps.

1. Chia predictor space—set of possible values của $X_1, X_2, \dots, X_p$ —thành J distinct, non-overlapping regions $R_1, R_2, \dots, R_J$.
2. Với mọi observation rơi vào region R_j , dùng cùng prediction: mean của response values cho training observations trong R_j .

Ví dụ, nếu Step 1 tạo hai regions R_1, R_2 , mean response ở first region là 10 và ở second region là 20, thì với observation $X = x$, ta predict 10 nếu $x \in R_1$ và 20 nếu $x \in R_2$.

Làm thế nào construct $R_1, \dots, R_J$? Về lý thuyết các regions có thể có shape bất kỳ, nhưng để đơn giản và dễ interpret, ta divide predictor space thành high-dimensional rectangles hay boxes. Goal là tìm boxes $R_1, \dots, R_J$ minimize RSS

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2, \quad (8.1)$$

trong đó $\hat{y}_{R_j}$ là mean response của training observations trong j th box. Không khả thi computationally khi xét mọi possible partition của feature space thành J boxes. Vì vậy ta dùng top-down, greedy approach gọi là *recursive binary splitting*. Approach là top-down vì bắt đầu ở top of tree, nơi mọi observations cùng thuộc một region, rồi successively split predictor space; mỗi split tạo hai branch mới. Nó là greedy vì ở mỗi step ta chọn best split ngay tại step đó, thay vì look ahead để chọn split có thể dẫn đến tree tốt hơn ở future step.

Để perform recursive binary splitting, đầu tiên chọn predictor X_j và cutpoint s sao cho split predictor space thành $\{X \mid X_j < s\}$ và $\{X \mid X_j \geq$

*recursive binary
splitting*

$s\}$ tạo reduction RSS lớn nhất. Ta xét mọi predictors $X_1, \dots, X_p$ và mọi possible cutpoint s , rồi chọn pair cho lowest RSS. Cụ thể, với j, s bất kỳ, define

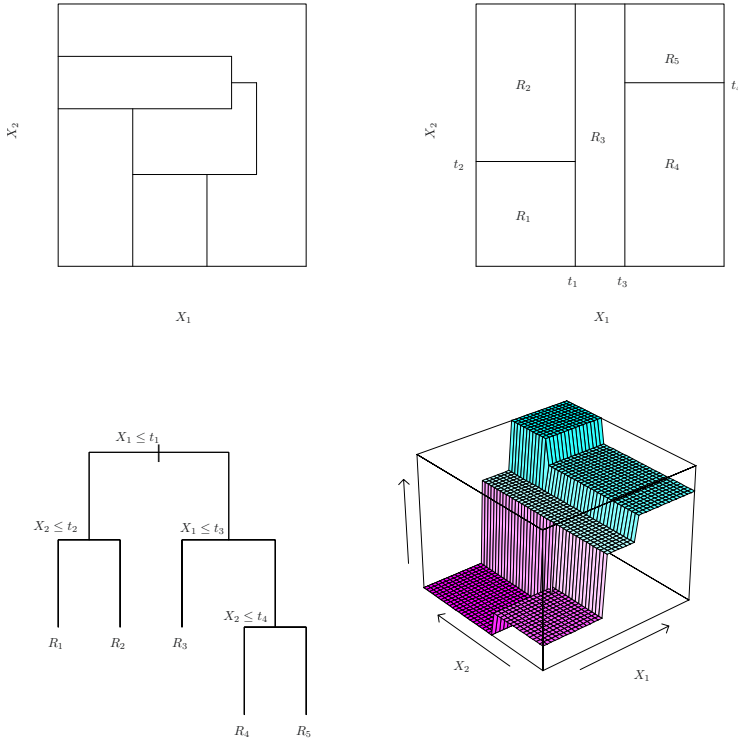
$$R_1(j, s) = \{X \mid X_j < s\} \quad \text{and} \quad R_2(j, s) = \{X \mid X_j \geq s\}, \quad (8.2)$$

và tìm j, s minimize

$$\sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2, \quad (8.3)$$

trong đó $\hat{y}_{R_1}$ và $\hat{y}_{R_2}$ là mean responses trong hai regions. Việc tìm j, s minimize (8.3) thực hiện khá nhanh, nhất là khi number of features p không quá lớn.

Tiếp theo repeat process, tìm best predictor và cutpoint để split data thêm sao cho minimize RSS trong resulting regions. Nhưng lần này chỉ split một trong hai regions đã có, tạo ba regions. Sau đó lại chọn một trong ba regions để split. Process tiếp tục đến stopping criterion; ví dụ, đến khi không region nào còn hơn năm observations.



HÌNH 8.3. Top Left: Một partition của two-dimensional feature space không thể sinh ra từ recursive binary splitting. Top Right: Output của recursive binary splitting trong một two-dimensional example. Bottom Left: Tree tương ứng với partition ở top right. Bottom Right: Perspective plot của prediction surface tương ứng tree đó.

Khi regions $R_1, \dots, R_J$ đã được tạo, response cho một test observation

được predict bằng mean của training observations trong region chứa test observation đó. Figure 8.3 minh họa example năm regions.

Tree Pruning

Process trên có thể predict tốt trên training set nhưng dễ overfit, dẫn đến poor test-set performance vì resulting tree quá complex. Một smaller tree với ít splits hơn, tức ít regions hơn, có thể giảm variance và tăng interpretability với cost là một chút bias. Có thể dùng grow tree khi reduction RSS từ mỗi split không vượt một high threshold, nhưng strategy này quá short-sighted: một split ban đầu có vẻ worthless có thể dẫn tới split rất tốt về sau.

Strategy tốt hơn là grow một very large tree T_0 , rồi prune lại để lấy subtree. Goal trực giác là chọn subtree có lowest test error. Với một subtree, có thể estimate test error bằng cross-validation hoặc validation set. Tuy nhiên, compute CV error cho mọi possible subtree quá cumbersome vì số subtree rất lớn. Ta cần một cách chọn small set of subtrees để xét.

Cost complexity pruning, còn gọi là *weakest link pruning*, làm đúng việc này. Thay vì mọi subtree, xét sequence of trees indexed bởi nonnegative tuning parameter α . Với mỗi α , có subtree $T \subset T_0$ sao cho

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \quad (8.4)$$

là nhỏ nhất. Ở đây $|T|$ là number of terminal nodes, R_m là rectangle tương ứng với m th terminal node, và $\hat{y}_{R_m}$ là predicted response của R_m , tức mean của training observations trong region đó. Parameter α control trade-off giữa complexity của subtree và fit với training data. Khi $\alpha = 0$, subtree chính là T_0 vì (8.4) chỉ measure training error. Khi α tăng, tree có nhiều terminal nodes bị penalty, nên smaller subtree có xu hướng minimize criterion. Equation 8.4 gọi nhớ lasso (6.7) ở Chapter 6, nơi formulation tương tự control complexity của linear model.

Khi tăng α từ zero, branches bị prune theo nested, predictable fashion, nên sequence of subtrees theo α dễ obtain. Ta chọn α bằng validation set hoặc cross-validation, rồi quay lại full data set và lấy subtree tương ứng. Process được summarize trong Algorithm 8.1.

Figures 8.4 và 8.5 cho results của fitting và pruning regression tree trên **Hitters** dùng chín features. Trước hết data được random split thành 132 training observations và 131 test observations. Sau đó grow large regression tree trên training data và vary α trong (8.4) để tạo subtrees với different numbers of terminal nodes. Cuối cùng dùng six-fold cross-validation để estimate CV MSE theo α .

Algorithm 8.1 Building a Regression Tree

1. Dùng recursive binary splitting để grow large tree trên training data, chỉ dừng khi mỗi terminal node có ít hơn một minimum number of observations.
2. Apply cost complexity pruning lên large tree để obtain sequence of best subtrees theo α .
3. Dùng K -fold cross-validation để chọn α . Chia training observations

prune
subtree

cost complexity
pruning
weakest link
pruning

thành K folds. Với mỗi $k = 1, \dots, K$:

- (a) Repeat Steps 1 và 2 trên tất cả training data trừ k th fold.
- (b) Evaluate mean squared prediction error trên left-out k th fold theo α .

Average results cho mỗi α và chọn α minimize average error.

4. Return subtree từ Step 2 tương ứng với chosen α .

Ta dùng six-fold CV vì 132 chia hết cho six. Unpruned regression tree được show ở Figure 8.4. Green curve trong Figure 8.5 là CV error theo number of leaves,² còn orange curve là test error; standard error bars cũng được show. Training error curve màu black làm reference. CV error là approximation hợp lý của test error: CV minimum tại three-node tree, còn test error cũng giảm mạnh ở three-node tree dù minimum thật nằm ở ten-node tree. Pruned tree với ba terminal nodes chính là Figure 8.1.

8.1.2 Classification Trees

Classification tree rất giống regression tree, trừ việc predict qualitative response thay vì quantitative response. Với regression tree, prediction cho observation là mean response của training observations cùng terminal node. Với classification tree, ta predict observation thuộc class xuất hiện nhiều nhất trong training observations của region đó. Khi interpret classification tree, ngoài class prediction ở mỗi terminal-node region ta thường quan tâm class proportions trong training observations của region.

Growing classification tree cũng gần như regression tree và vẫn dùng recursive binary splitting. Nhưng RSS không thể làm criterion cho binary splits. Một alternative tự nhiên là classification error rate. Vì observation ở region được assign vào most commonly occurring class, classification error rate là fraction training observations trong region không thuộc most common class:

$$E = 1 - \max_k (\hat{p}_{mk}). \quad (8.5)$$

Ở đây $\hat{p}_{mk}$ là proportion training observations trong m th region thuộc k th class. Tuy nhiên classification error không đủ sensitive để grow tree, nên thực tế thường dùng hai measures khác.

Gini index được định nghĩa là

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk}), \quad (8.6)$$

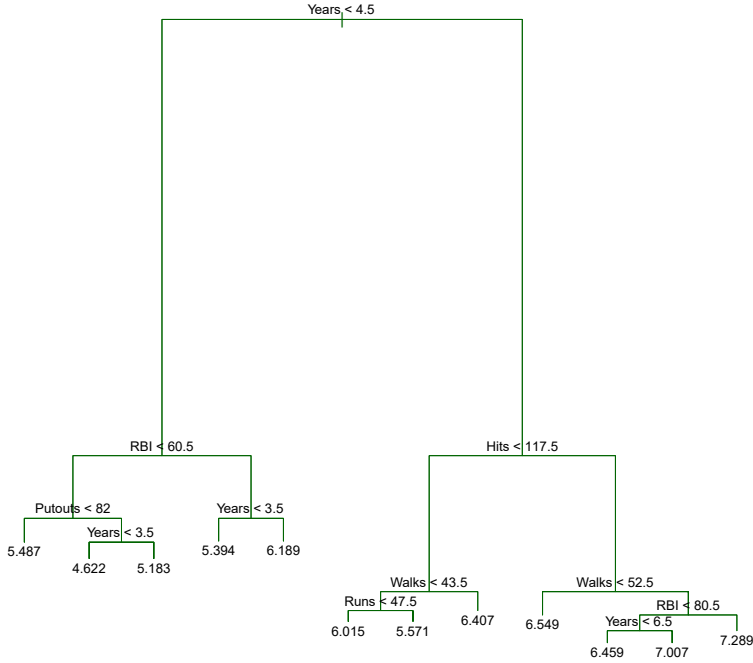
một measure của total variance trên K classes. Gini index nhỏ nếu mọi $\hat{p}_{mk}$ gần zero hoặc one; vì vậy nó là measure của *node purity*—small value nghĩa là node chủ yếu chứa observations từ một class.

²Dù CV error được compute theo α , tiện hơn khi display theo $|T|$, number of leaves; dựa trên relation giữa α và $|T|$ trong original tree grow từ toàn bộ training data.

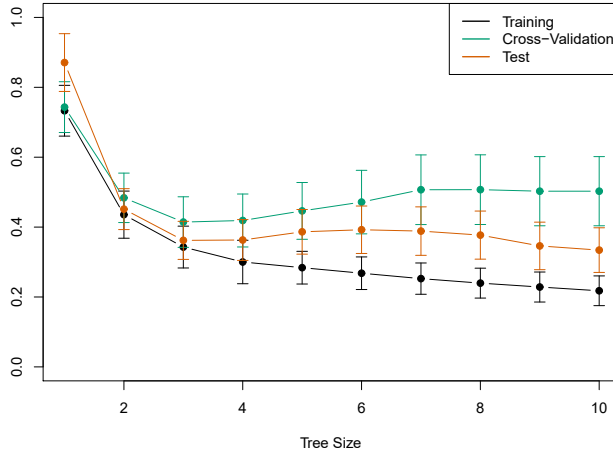
classification tree

classification error rate

Gini index



HÌNH 8.4. Regression tree analysis cho data *Hitters*. Unpruned tree sinh ra từ top-down greedy splitting trên training data.



HÌNH 8.5. Regression tree analysis cho data *Hitters*. Training, cross-validation và test MSE được show theo number of terminal nodes trong pruned tree. Standard error bands được display. Minimum cross-validation error xảy ra ở tree size bằng three.

Một alternative cho Gini index là entropy,

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}. \quad (8.7)$$

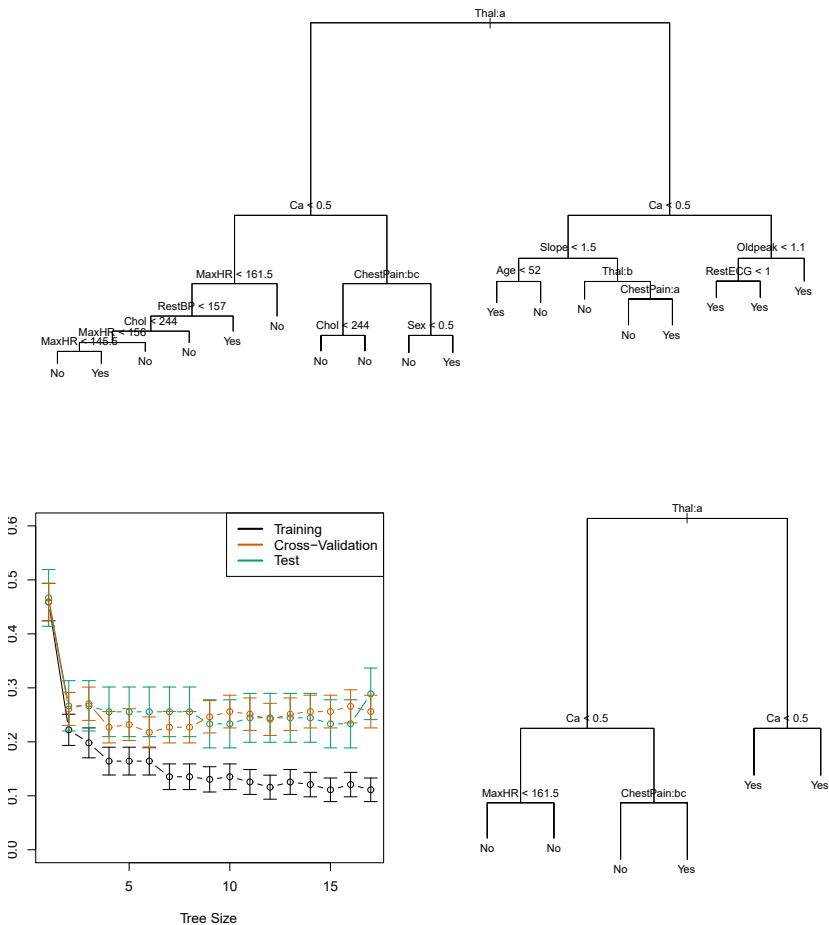
entropy

Vì $0 \leq \hat{p}_{mk} \leq 1$, nên $0 \leq -\hat{p}_{mk} \log \hat{p}_{mk}$. Entropy gần zero nếu các $\hat{p}_{mk}$ đều gần zero hoặc one. Vì thế, giống Gini index, entropy nhỏ khi m th node pure. Trên thực tế Gini index và entropy khá similar numerically.

Khi build classification tree, Gini index hoặc entropy thường được dùng để evaluate quality của split vì sensitive với node purity hơn classification error rate. Khi prune tree có thể dùng cả ba, nhưng nếu goal là prediction accuracy của final pruned tree thì classification error rate preferable.

Figure 8.6 minh họa trên data set **Heart**. Data có binary outcome **HD** cho 303 patients đến khám vì chest pain. Outcome **Yes** nghĩa là có heart disease theo angiographic test, **No** nghĩa là không. Có 13 predictors gồm **Age**, **Sex**, **Chol** và các heart/lung function measurements khác. Cross-validation cho tree với six terminal nodes.

Đến đây ta giả định predictors continuous, nhưng decision trees vẫn construct được với qualitative predictors. Trong **Heart**, một số predictors như **Sex**, **Thal** (Thallium stress test), và **ChestPain** là qualitative.



HÌNH 8.6. Data **Heart**. Top: Unpruned tree. Bottom Left: Cross-validation error, training error và test error cho different sizes của pruned tree. Bottom Right: Pruned tree tương ứng minimum cross-validation error.

Do đó, split trên qualitative variable tương đương assign một số qualitative values vào one branch và remaining values vào other branch. Trong Figure 8.6, top internal node split **Thal**. Text **Thal:a** nghĩa là left branch chứa observations với first value của **Thal** (normal), còn right branch chứa remaining observations (fixed hoặc reversible defects). Text **ChestPain:bc** ở hai splits phía dưới bên trái nghĩa là left branch chứa second và third values của **ChestPain**; possible values là typical angina, atypical angina, non-anginal pain và asymptomatic.

Figure 8.6 có characteristic hơi surprising: một số splits tạo hai terminal nodes với same predicted value. Ví dụ split **RestECG<1** gần bottom-right của unpruned tree. Bất kể **RestECG**, response **Yes** được predict. Split vẫn được thực hiện vì tăng node purity: toàn bộ 9 observations ở right-hand leaf là **Yes**, trong khi chỉ 7/11 observations ở left-hand leaf là **Yes**.

Tại sao node purity quan trọng? Nếu test observation nằm trong right-hand leaf đó, ta khá chắc response là **Yes**; nếu nằm trong left-hand leaf, response có lẽ là **Yes** nhưng uncertainty lớn hơn. Dù split **RestECG<1** không giảm classification error, nó cải thiện Gini index và entropy, vốn sensitive hơn với node purity.

8.1.3 Trees Versus Linear Models

Regression và classification trees có flavor rất khác các classical approaches trong Chapters 3 và 4. Linear regression assume model

$$f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j, \quad (8.8)$$

trong khi regression trees assume

$$f(X) = \sum_{m=1}^M c_m \cdot \mathbf{1}(X \in R_m), \quad (8.9)$$

trong đó $R_1, \dots, R_M$ là partition của feature space như Figure 8.3.

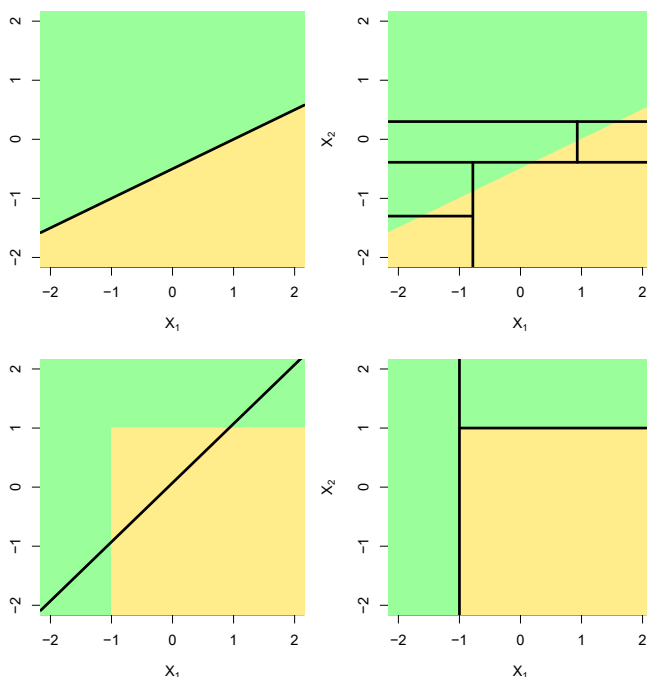
Model nào tốt hơn phụ thuộc problem. Nếu relationship giữa features và response được approximate tốt bởi linear model (8.8), linear regression thường work well và outperform regression tree vì tree không exploit linear structure. Nếu relationship highly non-linear và complex như (8.9), decision trees có thể outperform classical approaches. Figure 8.7 minh họa. Relative performance có thể assess bằng estimated test error qua cross-validation hoặc validation set approach (Chapter 5).

Ngoài test error còn các considerations khác khi chọn statistical learning method; chẳng hạn trong một số setting, prediction bằng tree được prefer vì interpretability và visualization.

8.1.4 Advantages and Disadvantages of Trees

Decision trees cho regression và classification có một số advantages so với classical approaches ở Chapters 3 và 4:

- ▲ Trees rất dễ explain cho người khác; thậm chí dễ explain hơn linear regression.



HÌNH 8.7. Top Row: Two-dimensional classification example với true decision boundary linear, thể hiện bằng shaded regions. Classical approach assume linear boundary (left) outperform decision tree với splits parallel to axes (right). Bottom Row: True decision boundary non-linear. Linear model không capture được boundary (left), còn decision tree thành công (right).

- ▲ Một số người cho rằng decision trees gần với human decision-making hơn regression/classification approaches ở các chapter trước.
- ▲ Trees có thể display graphically và dễ interpret ngay cả với non-expert, nhất là khi tree nhỏ.
- ▲ Trees handle qualitative predictors dễ dàng mà không cần create dummy variables.
- ▼ Tiếc là trees thường không đạt predictive accuracy ngang một số regression/classification approaches khác trong sách.
- ▼ Trees cũng có thể rất non-robust: small change trong data có thể gây large change trong final estimated tree.

Tuy nhiên, bằng cách aggregate nhiều decision trees qua bagging, random forests và boosting, predictive performance có thể cải thiện đáng kể. Các concepts này được giới thiệu ở section tiếp theo.

8.2 Bagging, Random Forests, Boosting, and Bayesian Additive Regression Trees

Ensemble method là approach combine nhiều simple “building block” models để obtain một single model có thể rất powerful. Các simple building blocks đôi khi được gọi là *weak learners*, vì từng model riêng lẻ chỉ có thể cho mediocre predictions.

*ensemble
weak learners*

Ta sẽ discuss bagging, random forests, boosting và Bayesian additive regression trees. Đây là ensemble methods mà building block đơn giản là regression tree hoặc classification tree.

8.2.1 Bagging

Bootstrap, giới thiệu ở Chapter 5, là một idea cực kỳ powerful. Nó được dùng trong nhiều setting khi standard deviation của quantity cần quan tâm khó hoặc thậm chí không thể compute trực tiếp. Ở đây bootstrap được dùng theo context hoàn toàn khác: cải thiện statistical learning methods như decision trees.

Decision trees ở Section 8.1 có high variance. Nếu random split training data thành hai parts rồi fit decision tree trên mỗi half, results có thể rất khác nhau. Ngược lại, procedure có low variance cho similar results khi repeatedly apply lên distinct data sets; linear regression thường có low variance nếu ratio n/p moderately large. *Bootstrap aggregation*, hay *bagging*, là general-purpose procedure để reduce variance của statistical learning method; nó đặc biệt useful và commonly used với decision trees.

bagging

Nhớ rằng với n independent observations $Z_1, \dots, Z_n$, mỗi observation có variance σ^2 , variance của mean $\bar{Z}$ bằng σ^2/n . Nói cách khác, averaging observations làm giảm variance. Vì thế một natural way để giảm variance và tăng test-set accuracy là lấy nhiều training sets từ population, build separate prediction model trên mỗi training set, rồi average predictions. Nếu có B separate training sets, ta có thể compute $\hat{f}_1(x), \dots, \hat{f}_B(x)$ và lấy

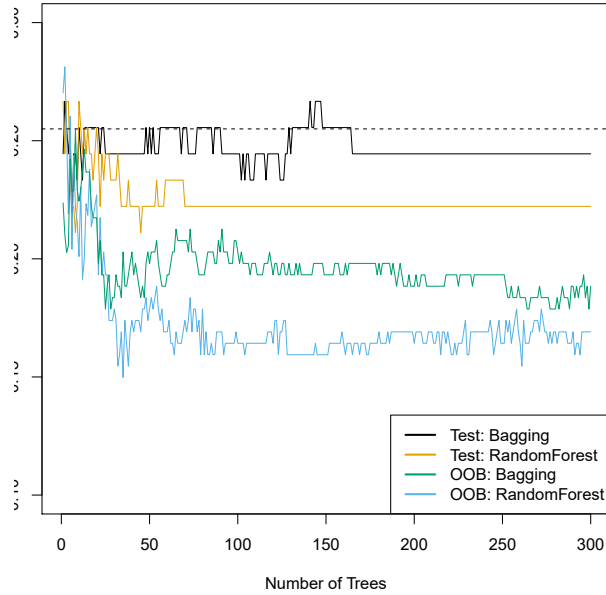
$$\hat{f}_{\text{avg}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

Dĩ nhiên điều này không practical vì thường chỉ có một training set. Thay vào đó, ta bootstrap bằng repeated samples từ single training data set. Generate B bootstrapped training data sets; train method trên b th bootstrapped set để có $\hat{f}_b^*(x)$, rồi average:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b^*(x).$$

Đây chính là bagging.

Bagging có thể improve predictions cho nhiều regression methods, nhưng đặc biệt useful với decision trees. Với regression trees, construct B regression trees từ B bootstrapped training sets rồi average predictions. Các trees này được grow deep và không prune, nên mỗi individual tree có high variance nhưng low bias; averaging B trees làm giảm variance. Bagging đã



HÌNH 8.8. Bagging và random forest results cho data **Heart**. Test error (black và orange) được show theo B , number of bootstrapped training sets. Random forests dùng $m = \sqrt{p}$. Dashed line là test error từ một single classification tree. Green và blue traces là OOB error, tình cờ thấp hơn đáng kể trong example này.

cho improvements ấn tượng bằng cách combine hàng trăm hoặc hàng nghìn trees thành một procedure duy nhất.

Đến đây bagging được mô tả cho regression, với quantitative outcome Y . Với classification, response Y qualitative, cách đơn giản nhất là ghi lại class predicted bởi mỗi trong B trees cho một test observation rồi lấy *majority vote*: overall prediction là class xuất hiện nhiều nhất trong B predictions.

Figure 8.8 show results từ bagging trees trên data **Heart**. Test error rate được show theo B , number of trees built từ bootstrapped training data. Bagging test error hơi thấp hơn single-tree test error trong case này. Number of trees B không phải critical parameter: dùng B rất lớn không gây overfitting. Trong practice, chọn B đủ lớn để error đã settle down; $B = 100$ đủ cho good performance trong example.

Out-of-Bag Error Estimation

Có một cách rất straightforward để estimate test error của bagged model mà không cần cross-validation hay validation set. Key của bagging là trees repeated fit trên bootstrapped subsets. Trung bình mỗi bagged tree dùng khoảng two-thirds observations.³ One-third còn lại không dùng để fit tree được gọi là *out-of-bag (OOB) observations*.

Ta có thể predict response cho i th observation bằng từng tree trong đó observation đó là OOB, tạo khoảng $B/3$ predictions. Để có single prediction, average predicted responses nếu là regression hoặc take majority vote nếu là classification. Làm vậy cho mọi n observations rồi compute overall

³Điều này liên quan tới Exercise 2 của Chapter 5.

majority vote

out-of-bag

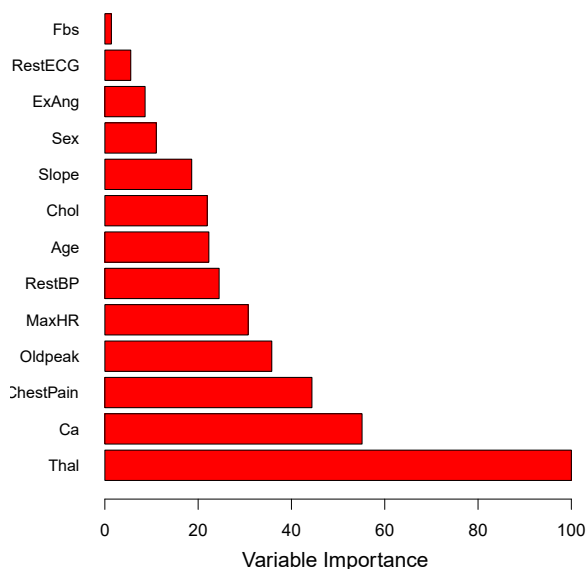
OOB MSE hoặc classification error. OOB error là valid estimate của test error vì response của mỗi observation chỉ được predict bởi trees không dùng observation đó khi fit.

Figure 8.8 display OOB error trên data **Heart**. Với B đủ lớn, OOB error gần như equivalent với leave-one-out cross-validation error. OOB estimation đặc biệt convenient khi bagging trên large data sets nơi cross-validation computationally onerous.

Variable Importance Measures

Bagging thường tăng accuracy so với single tree, nhưng model result khó interpret. Một advantage của decision tree là diagram attractive và dễ hiểu như Figure 8.1. Khi bag rất nhiều trees, không thể represent whole procedure bằng một tree duy nhất, và không còn obvious variable nào important. Vì vậy bagging improve prediction accuracy với cost là interpretability.

Dù collection of bagged trees khó interpret hơn single tree, ta vẫn có overall summary của importance cho mỗi predictor qua RSS (bagging regression trees) hoặc Gini index (bagging classification trees). Với regression trees, record total amount RSS trong (8.1) giảm do splits trên một predictor, average trên B trees; large value chỉ important predictor. Với classification trees, cộng total amount Gini index (8.6) giảm do splits trên predictor đó, rồi average trên B trees.



HÌNH 8.9. Variable importance plot cho data **Heart**. Variable importance được compute bằng mean decrease in Gini index và biểu diễn relative to maximum.

Graphical representation của variable importances trong data **Heart** được show ở Figure 8.9. Ta thấy mean decrease in Gini index cho mỗi variable, relative to largest. Các variables có largest mean decrease in Gini index là Thal, Ca, và ChestPain.

variable importance